



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ Информатика и системы управления (ИУ)

КАФЕДРА _____ Теоретическая информатика и компьютерные технологии (ИУ9)

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2
по курсу «Алгоритмы компьютерной графики»
«Модельно-видовые и проективные преобразования»

Выполнила: *Зайковская*
Екатерина Владимировна,
ИУ9-42Б

Москва 2023

Задача

Главной задачей лабораторной работы является реализация модельно-видовых и проективных преобразований. Для этого сформированы следующие пункты задания:

1. Определить куб в качестве модели объекта сцены.
2. Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта).
3. Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без `gluLookAt`). Управление производится интерактивно с помощью клавиатуры и/или мыши.
4. Предусмотреть возможность переключения между каркасным и твердотельным отображением модели.

Важное пояснение к условию: *запрещено использование встроенных функций геометрических преобразований (таких как `glScaled()`, `glTranslated()` и др.).*

Индивидуальный вариант заключается в реализации **двухточечной перспективы**.

Основная теория

Преобразование модели

Итоговое изображение фигуры формируется **композицией преобразований**. Модельные и видовые преобразования в библиотеке OpenGL объединены (для простоты), они используются для «перемещений» и «трансформаций» исходного объекта. Проективные преобразования схожи со взглядом камеры - они позволяют перевести трехмерную модель в двумерное изображение, добавив, например, перспективные искажения.

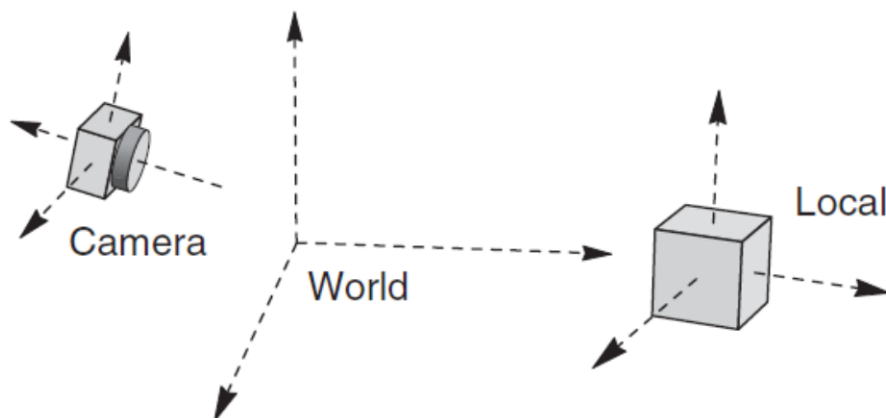


Рис.: Системы координат сцены с фигурой

Таким образом:

Модельное преобразование:

- переводит модель, заданную в локальных (собственных) координатах, в глобальное (мировое пространство);
- сцена «собирается» из частей, с помощью модельных преобразований (обычно композиция переносов, поворотов, масштабирования);
- на выходе – модель в единых мировых координатах.

Видовое преобразование:

- «подгоняет» мир под стандартную камеру, преобразует мировую систему координат в видовые координаты (которые подходят для «стандартной» камеры);
- на выходе – модель, готовая к проекции на экран.

Проективное преобразование:

- реализует 3D преобразование, подготавливая модель к переходу к 2D (координата z сохраняется);
- проекция - переход к двумерной системе координат;
- после проективного преобразования отбрасывается координата z и получаются значения в оконных координатах;
- проективное преобразование ограничивает видимую область пространства.

Каждое *геометрическое преобразование представляется в виде матрицы*, и перевод координат из начальной системы координат в новую производится умножением:

- Координаты представляются вектором-столбцом.
- Геометрическое преобразование задается матрицей, умножаемой справа на вектор-столбец координат:

$$\mathbf{P}' = \mathbf{M} * \mathbf{P}$$

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ m & n & p & q \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- Матрица композиции преобразований является произведением матриц элементарных преобразований (матрицы перемножаются в обратном порядке).
- Обратное преобразование задается обратной матрицей.

Задание матриц преобразований, необходимых для варианта лабораторной работы

Согласно заданию необходимо реализовать поворот куба и перспективную проекцию. Для этого и для корректного отображения модели использовались перечисленные ниже конкретные типы преобразований.

Модельно-видовые преобразования

Трехмерный поворот (относительно осей X и Y)

$$R_{Ox} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \varphi & -\sin \varphi & 0 \\ 0 & \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_{Oy} = \begin{pmatrix} \cos \varphi & 0 & \sin \varphi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \varphi & 0 & \cos \varphi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

φ - угол поворота

Трёхмерное масштабирование

$$x' = ax$$

$$y' = by$$

$$z' = cz$$

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} a & 0 & 0 & 0 \\ 0 & b & 0 & 0 \\ 0 & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

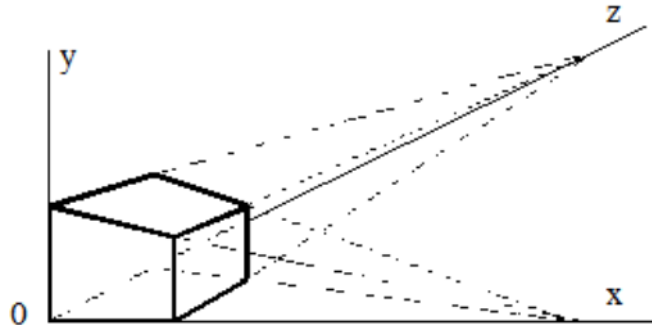


a, b, c - коэффициенты масштабирования по осям X, Y, Z

Проективные преобразования

Центральная (перспективная) двухточечная проекция

$$\begin{pmatrix} 1 & 0 & 0 & -1/a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1/c \\ 0 & 0 & 0 & 1 \end{pmatrix}^T$$



a, c - координаты точек схода по осям X и Z

Практическая реализация

Прим: язык реализации: C++.

Для применения преобразований были созданы функции, аналогичные встроенным: поворот вокруг осей X, Y , масштабирование - и функция двухточечной перспективы. (Матрицы записываются по строкам, т.е. в транспонированном виде.)

```
1 void rotate_ox(GLfloat phi){  
2     GLfloat m_rotation_x[] = {1, 0, 0, 0,
```

```

3             0, cosf(phi), sinf(phi), 0,
4             0, -1*sinf(phi), cosf(phi), 0,
5             0, 0, 0, 1};
6     glMultMatrixf(m_rotation_x);
7 }
8
9 void rotate_oy(GLfloat phi){
10    GLfloat m_rotation_y[] = {cosf(phi), 0, -1*sinf(phi), 0,
11                               0, 1, 0, 0,
12                               sinf(phi), 0, cosf(phi), 0,
13                               0, 0, 0, 1};
14    glMultMatrixf(m_rotation_y);
15 }
16
17 void scale(GLfloat x, GLfloat y, GLfloat z){
18    GLfloat m_scale[] = {x, 0, 0, 0,
19                         0, y, 0, 0,
20                         0, 0, z, 0,
21                         0, 0, 0, 1};
22    glMultMatrixf(m_scale);
23 }
24
25 void two_p_persp(){
26    GLfloat m_persp[] = {1, 0, 0, coef_persp,
27                         0, 1, 0, 0,
28                         0, 0, 1, coef_persp,
29                         0, 0, 0, 1};
30    glMultMatrixf(m_persp);
31 }

```

Управление кубом внутри сцены производится:

- клавишами 'A', 'D', 'W', 'S' - вращение вокруг осей X и Y;
- клавишами '+' и '-' - приближение и удаление точек схода перспективы;
- клавишей 'V' для переключения каркасного и твердотельного отображения.

Последний пункт осуществляется переключением режима отображения граней (GL_LINE / GL_FILL):

```

1 void display() {
2     ...
3     if (!polygon_mode) glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
4     else glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
5     ...
6 }

```

В результате работы программы в окне отображаются 2 куба, вращающихся под одним углом, но при этом к одному (тому, что больше) применяется перспективное преобразование, к другому нет.

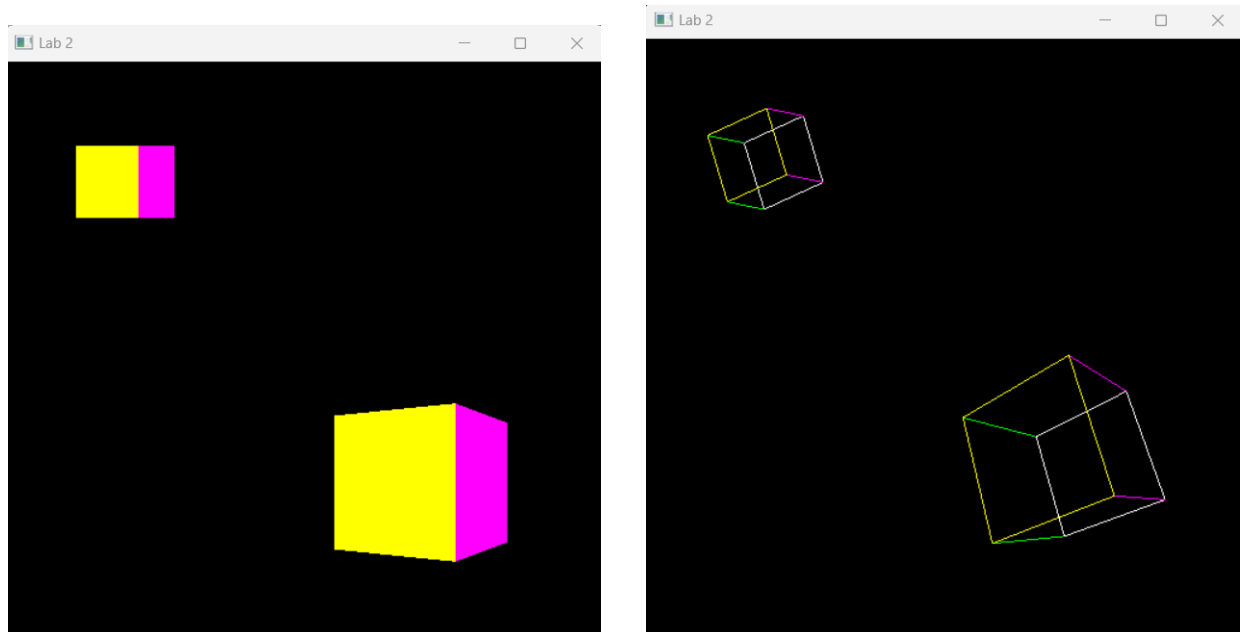


Рис.: Окно программы

Такое отображение возможно благодаря созданию двух протов вывода, в каждом из которых задаются свои преобразования.

```
1 static int min_w_h;
2 ...
3 void reshape(GLint w, GLint h) {
4     if (w > h) min_w_h = h;
5     else min_w_h = w;
6 }
7
8 void display() {
9     glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
10
11     if (!polygon_mode) glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
12     else glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
13
14     glViewport(0.4*min_w_h, 0, 0.6*min_w_h, 0.6*min_w_h);
15     //drawing a cube with perspective
16     ...
17
18     glViewport(0, 0.6*min_w_h, 0.4*min_w_h, 0.4*min_w_h);
19     //drawing a simple cube
20     ...
21
```

```
22  glFlush () ;  
23  glutSwapBuffers () ;  
24 }
```

Это основные особенности реализации, в остальном применяются стандартные приемы отрисовки полигонов, обработки нажатий клавиш (аналогично предыдущей лабораторной работе). Код полностью приводится в приложении.

Заключение

В ходе лабораторной работы было создано оконное приложение (на основе библиотеки OpenGL), демонстрирующее разницу между изображениями куба с использованием перспективного преобразования и без.

Для этого были использованы теоретические основы геометрических преобразований, а именно их матричное представление и запись преобразований координат при переходе от одного базиса к другому. А также использовались функции умножения матриц библиотеки OpenGL для применения заданных трансформаций.

Полный код реализации*язык C++*

```
1 #define _USE_MATH_DEFINES
2 #include <GL/glut.h>
3 #include <cmath>
4
5 static bool polygon_mode = true;
6 static GLfloat coef_persp = 0.15;
7 static GLfloat r_x = 0.0;
8 static GLfloat r_y = 0.0;
9 static int min_w_h;
10
11
12 void draw_cube() {
13     glBegin(GL_POLYGON);
14     glColor3f( 1.0, 1.0, 0.0 );
15     glVertex3f( 0.5, -0.5, -0.5 );
16     glVertex3f( 0.5, 0.5, -0.5 );
17     glVertex3f( -0.5, 0.5, -0.5 );
18     glVertex3f( -0.5, -0.5, -0.5 );
19     glEnd();
20
21     glBegin(GL_POLYGON);
22     glColor3f( 1.0, 1.0, 1.0 );
23     glVertex3f( 0.5, -0.5, 0.5 );
24     glVertex3f( 0.5, 0.5, 0.5 );
25     glVertex3f( -0.5, 0.5, 0.5 );
26     glVertex3f( -0.5, -0.5, 0.5 );
27     glEnd();
28
29     glBegin(GL_POLYGON);
30     glColor3f( 1.0, 0.0, 1.0 );
31     glVertex3f( 0.5, -0.5, -0.5 );
32     glVertex3f( 0.5, 0.5, -0.5 );
33     glVertex3f( 0.5, 0.5, 0.5 );
34     glVertex3f( 0.5, -0.5, 0.5 );
35     glEnd();
36
37     glBegin(GL_POLYGON);
38     glColor3f( 0.0, 1.0, 0.0 );
39     glVertex3f( -0.5, -0.5, 0.5 );
40     glVertex3f( -0.5, 0.5, 0.5 );
41     glVertex3f( -0.5, 0.5, -0.5 );
42     glVertex3f( -0.5, -0.5, -0.5 );
43     glEnd();
```

```

44
45 glBegin(GL_POLYGON);
46 glColor3f( 0.0, 0.0, 1.0 );
47 glVertex3f( 0.5, 0.5, 0.5 );
48 glVertex3f( 0.5, 0.5, -0.5 );
49 glVertex3f( -0.5, 0.5, -0.5 );
50 glVertex3f( -0.5, 0.5, 0.5 );
51 glEnd();
52
53 glBegin(GL_POLYGON);
54 glColor3f( 1.0, 0.0, 0.0 );
55 glVertex3f( 0.5, -0.5, -0.5 );
56 glVertex3f( 0.5, -0.5, 0.5 );
57 glVertex3f( -0.5, -0.5, 0.5 );
58 glVertex3f( -0.5, -0.5, -0.5 );
59 glEnd();
60 }
61
62 void rotate_ox(GLfloat phi){
63     GLfloat m_rotation_x[] = {1, 0, 0, 0,
64                                0, cosf(phi), sinf(phi), 0,
65                                0, -1*sinf(phi), cosf(phi), 0,
66                                0, 0, 0, 1};
67     glMultMatrixf(m_rotation_x);
68 }
69
70 void rotate_oy(GLfloat phi){
71     GLfloat m_rotation_y[] = {cosf(phi), 0, -1*sinf(phi), 0,
72                                0, 1, 0, 0,
73                                sinf(phi), 0, cosf(phi), 0,
74                                0, 0, 0, 1};
75     glMultMatrixf(m_rotation_y);
76 }
77
78 void scale(GLfloat x, GLfloat y, GLfloat z){
79     GLfloat m_scale[] = {x, 0, 0, 0,
80                           0, y, 0, 0,
81                           0, 0, z, 0,
82                           0, 0, 0, 1};
83     glMultMatrixf(m_scale);
84 }
85
86 void two_p_persp(){
87     GLfloat m_persp[] = {1, 0, 0, coef_persp,
88                           0, 1, 0, 0,
89                           0, 0, 1, coef_persp,

```

```

90         0, 0, 0, 1};
91     glMultMatrixf(m_persp);
92 }
93
94 void reshape(GLint w, GLint h) {
95     if (w > h) min_w_h = h;
96     else min_w_h = w;
97 }
98
99 void display() {
100     glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);
101
102     if (!polygon_mode) glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
103     else glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
104
105     glViewport(0.4*min_w_h, 0, 0.6*min_w_h, 0.6*min_w_h);
106     glMatrixMode(GL_MODELVIEW);
107     glLoadIdentity();
108     rotate_ox(r_x);
109     rotate_oy(r_y);
110     glMatrixMode(GL_PROJECTION);
111     glLoadIdentity();
112     scale(0.7, 0.7, 0.7);
113     rotate_oy(-M_PI/4);
114     two_p_persp();
115     draw_cube();
116
117     glViewport(0, 0.6*min_w_h, 0.4*min_w_h, 0.4*min_w_h);
118     glMatrixMode(GL_MODELVIEW);
119     glLoadIdentity();
120     rotate_ox(r_x);
121     rotate_oy(r_y);
122     glMatrixMode(GL_PROJECTION);
123     glLoadIdentity();
124     scale(0.6, 0.6, 0.6);
125     rotate_oy(-M_PI/4);
126     draw_cube();
127
128     glFlush();
129     glutSwapBuffers();
130 }
131
132
133 void press(unsigned char button, int x, int y) {
134     if (button == 'a') r_y += 0.05;
135     else if (button == 'd') r_y -= 0.05;

```

```

136     else if (button == 'w') r_x += 0.05;
137     else if (button == 's') r_x -= 0.05;
138     else if (button == 'v') polygon_mode = (polygon_mode + 1) % 2;
139     else if (button == '+'){
140         if (coef_persp < 0.4) coef_persp += 0.03;
141     }
142     else if (button == '-'){
143         if (coef_persp >= 0.03) coef_persp -= 0.03;
144     }
145     glutPostRedisplay();
146 }
147
148
149 int main(int argc, char** argv) {
150     glutInit(&argc, argv);
151     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
152     glutInitWindowPosition(80, 80);
153     glutInitWindowSize(500, 500);
154     glutCreateWindow("Lab 2");
155     glEnable(GL_DEPTH_TEST);
156     glutReshapeFunc(reshape);
157     glutDisplayFunc(display);
158     glutKeyboardFunc(press);
159     glutMainLoop();
160 }

```